Retinal Screening and Referral Support: Final Product Technical Report

Michael Kimani · BSc Software Engineering (ML), African Leadership University · Supervisor: Emmanuel Adjei · 27 October 2026
Repository: github.com/Mikekimm/michael-kimani-eye-screening-capstone (branch `main`) · Video: **[link]**

Scope of claims. This is screening and referral *support*, not a diagnostic device. The models were trained and evaluated only on public datasets; there has been no field deployment, external validation or clinical validation. Every figure below comes from a recorded run in the repository.

1. Scope alignment and feature validation

The approved proposal planned an offline-first Android app that screens fundus photos for diabetic retinopathy (DR), glaucoma and cataract on the device, refers uncertain or high-risk cases to a specialist dashboard, and syncs when connectivity returns. Following supervisor feedback (a feasible prototype with one demonstrable workflow; reconcile offline use with sync and authentication; separate research evaluation from development), effort went first into one complete, tested workflow; the model registry then made adding the glaucoma and cataract models a data task with no code change. The platform changed from React Native with on-device TensorFlow Lite to an installable PWA with server-side inference (one code base for Android and iPhone, models verified and replaced on the server); offline therefore covers capture and queueing, and scoring happens after sync. The full table with evidence paths is in `docs/scope_table.md`.

Planned feature	Intended purpose	Status	Evidence
DR classifier (MobileNetV2, TFLite)	Detect DR from a fundus photo	Completed	v2.0 fine-tuned; `notebooks/dr_v2_finetune.ipynb`; recomputation and parity tests
Glaucoma and cataract classifiers	Multi-disease screening	Completed	ODIR-5K, patient-separated split; glaucoma below target (precision 0.103), cataract limited evidence (46 positives)
Five-band cataract grading	Grade cataract severity	Not implemented	ODIR-5K has no severity labels; glaucoma and cataract are binary only rather than showing invented bands
Risk / severity output	Show urgency	Changed	DR: five-band *estimated* band from the binary score (`RiskPolicy`, boundary tests); others binary
Sensitivity ≥ 85%, specificity ≥ 80%	Safe screening trade-off (RQ3)	Partial	DR test split: sensitivity 0.851 met, specificity 0.748 not met; glaucoma not met; cataract met
On-device inference	Screen without internet	Changed	Server-side LiteRT; offline = capture and queue, scored after sync
Android app (React Native)	Mobile client	Changed	Installable PWA for Android and iPhone; E2E on both engines' emulation
Register patient, capture image, view result	Core health-worker workflow	Completed	E2E journey 1, screenshots
Offline storage and two-way sync	Work with intermittent connectivity	Completed	IndexedDB queue, idempotent push, decision pull; 10 sync tests
Flag and refer uncertain / high-risk cases	Structured referral	Completed	Automatic and manual referral, workflow tests
Specialist review: approve or override	Close the referral loop	Completed	Override any condition with a reason, retake, comment; waiting time and overdue targets; E2E
Admin: users and analytics	Access control, patterns	Completed	Approve / reject sign-ups, staff accounts, facility list, aggregate analytics
FastAPI + PostgreSQL backend	Central store and API	Completed	SQLite locally; backend suite on PostgreSQL 16 and Docker build green in GitHub Actions
Load balancer	Scale to many clinics	Not implemented	Stateless JWT API can scale horizontally
Sync tested under interruptions	Reliability (objective 4)	Completed	Lost-reply, offline-queue and backoff tests
Usability test with health workers	Check usability	Not yet	SUS study planned

Added beyond the proposal: sign-up that stays **pending** until an administrator approves it (facility chosen from an admin-managed list; the registration number is recorded for the administrator to check, never claimed as verified; rejection needs a reason), role-based access checked on every request (Argon2, JWT refresh rotation, sign-in and sign-up rate limits), an image quality gate, a SHA-256-verified model registry whose cards also state each model's resize, eye laterality and model version on every result, an append-only review log and an audit log.

2. Technical functionality

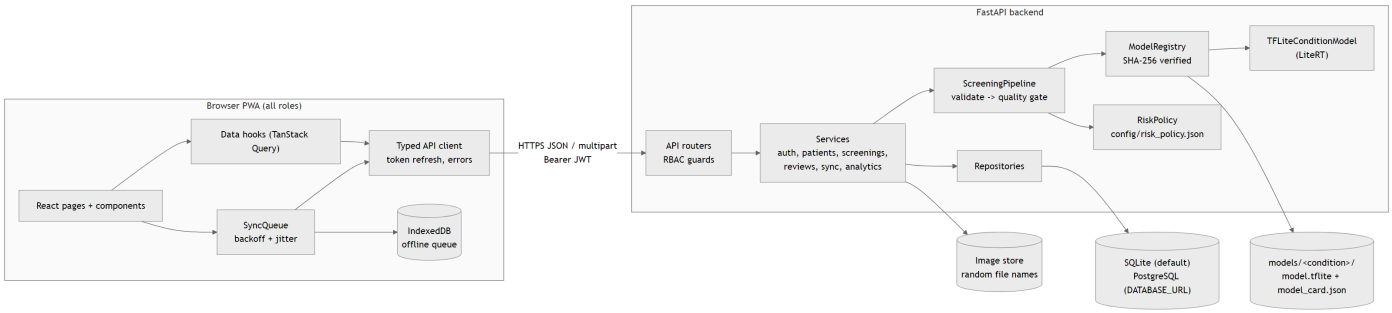


Figure 1. Architecture as implemented (generated from `docs/diagrams/src/architecture.mmd`).

Frontend. A React 18 + TypeScript (strict) PWA, mobile-first (bottom tabs below 1024 px, sidebar above). Pages read server data only through hooks that call one typed API client, which attaches the JWT, refreshes it once on a 401 for all concurrent requests, and converts failures into plain-language messages. Health workers register patients by clinic code, choose the eye, and capture a photo with a circular guide (rear camera or gallery); on-device hints flag dark or blurry photos before upload.

Backend. FastAPI with a layered design: routers (HTTP and role guards) → services (use cases) → repositories (all SQL). `POST /api/screenings` checks type, size and that the bytes really decode as JPEG/PNG, applies the quality gate (blank or tiny images return 422 with a reason), then runs every model the registry has verified and maps each score through that condition's risk policy. One `diagnosis_results` row is stored per condition with its model version, including "not available" rows, so history shows exactly what ran. Positive or borderline results open a referral with a review target (urgent 24 h, routine 72 h; a configurable design decision, since the proposal sets no limit); overdue cases rise to the top of the specialist queue. Specialists approve, override any scored condition (reason required), request a retake or comment; each action is appended to `review_events`, which the ORM refuses to update or delete, and reaches the health worker on the next sync. Each result card shows the model's measured limits; results made by an earlier model are never re-scored and carry a notice instead.

Offline and sync. Without a connection, photos and new patients are stored in IndexedDB with a client-generated UUID. A sync engine pushes them in order (patients first), retrying with bounded exponential backoff and full jitter; because the server is idempotent on the UUID, a retry after a lost reply returns the original record rather than a duplicate. Each cycle then pulls specialist decisions made since the last pull. Offline mode captures and queues; it does not run the model on the device.

Data handling and privacy. SQLite by default, PostgreSQL by configuration, schema managed by Alembic migrations. Patients are pseudonymous codes; images are stored under random names and served only to authorised users; analytics and audit records contain no images or patient fields; secrets live only in a git-ignored `.env` generated by a script.

3. Code quality, OOP and algorithms

Principle / pattern	Where
Abstraction + polymorphism	`ConditionModel` → `TFLiteConditionModel`; tests substitute a labelled double
Strategy	`RiskPolicy` per condition, built by `RiskPolicyFactory` from JSON
Open/closed	`ModelRegistry`: new condition = new folder
Repository + DI	backend repositories; `QueueRepository` (IndexedDB / in-memory); `api/deps.py`
Encapsulation	append-only history enforced by ORM listeners; one `AccessPolicy` for record visibility
Single responsibility	routers, services, repositories, presenters, schemas in separate modules

Algorithm	Purpose and complexity
Laplacian variance, brightness, vignette test	quality gate, O(pixels) on a 256^2 copy
Sensitivity-targeted threshold search	highest threshold meeting 85% sensitivity, O(n log n) via sort + binary search
Rank-sum AUC, stratified bootstrap CIs	honest evaluation, NumPy only; AUC matches scikit-learn
Threshold-anchored severity bands	bands can never contradict the decision (1,001-score sweep test)
Idempotent sync, backoff with full jitter	no duplicates, no retry storms
Streaming SHA-256; per-model resize	model integrity; the input each model was trained with

Quality practices: type hints throughout the backend and TypeScript strict mode with ESLint (which forbids `fetch` outside the API client); Pydantic v2 schemas separate from ORM entities; centralised configuration (settings, risk policy, model cards); small conventional commits on a feature branch (`git log`); a reproducible environment (pinned requirements, `package-lock.json`). Superseded Streamlit code was removed and tagged in git rather than left alongside.

4. Testing approaches and results

Strategy	Why chosen	How	Result
Unit (algorithms)	boundary errors would mis-triage silently	pytest, Vitest; boundary values, property sweep; deliberate code breakage to check tests fail	119 + 69 pass

Integration (API + DB + real model)	security and integrity rules live between layers	fresh DB per test; real LiteRT models; 56-case role matrix; approval rules; PostgreSQL in CI	136 pass
End-to-end functional	prove users can finish clinical tasks	Playwright: fake sign-up refused until approved, rejection, worker journey, specialist review, unauthorized access, invalid input	24/24 pass
Compatibility	clinics use varied phones and browsers	same journeys on Chromium (Pixel 5, desktop), Firefox, WebKit (iPhone 13)	all engines pass
Performance	slow hardware and networks	benchmark script; throttled browser round trip; Lighthouse	see section 5
Accessibility	low-vision and screen-reader users	axe-core WCAG 2.1 A/AA on 20 pages, all roles, mobile and desktop	0 violations
Security (brute force)	password guessing	sliding-window login throttle; unit tests with a fake clock and API tests	429 after 5 failures
Model validation and parity	RQ3; the app must feed models as trained	held-out test splits; metrics recomputed with the app's code; 24-photo parity	see section 6

Different inputs: negative, borderline and positive scores, real and synthetic fundus photos, blank, tiny, corrupt, mislabelled, oversized and future-dated uploads, name-like patient codes, free-text facilities, weak passwords, expired and replayed tokens, and every role against every route. **Improvements made because of testing:** a form test exposed that the password field dropped its form reference, so sign-in could never succeed (fixed with `forwardRef`); an end-to-end script test exposed that the calibration script did not write its outputs (fixed); screenshots led to listing DR first and to plain-language referral reasons; Lighthouse raised SEO from 91 to 100; axe-core found unlabeled camera inputs and, after the new models, low-contrast amber badges (both fixed); the parity test showed the app was resizing every photo with nearest-neighbour, which would have changed 6 of 24 DR v2 decisions, so each model card now states its resize.

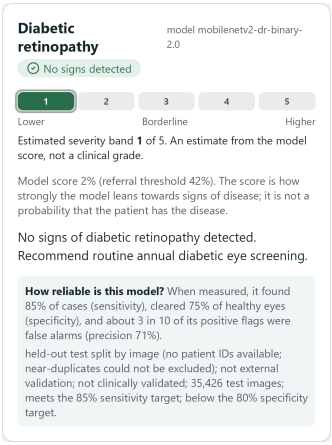


Figure 2. DR result card on iPhone 13 (WebKit), from a synthetic test image: estimated band, model version, score explained, action.

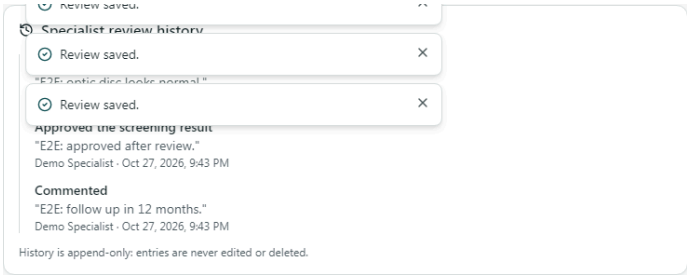


Figure 3. Specialist review history (append-only), desktop Chrome.

5. Performance and compatibility

Measurement (Windows 11, Intel CPU, 8 logical cores)	Result
Model inference, LiteRT, 200 runs (all cores / 1 thread), DR v1 2.6 MB model	55 / 60 ms mean, 68 / 84 ms p95; about 117 MB process memory (not yet re-run with the three 9 MB models)
Full server pipeline on a 1024 px JPEG (all cores / 1 thread)	74 / 77 ms mean (an earlier run under background load: 166 ms)
Upload to result in the browser (median of 3)	0.50 s; 3.17 s with emulated 6x slower CPU and slow 3G
Lighthouse, sign-in page (performance / accessibility / best practices / SEO)	mobile 96 / 100 / 100 / 100; desktop 100 / 100 / 100 / 100
Accessibility of signed-in screens (axe-core, WCAG 2.1 AA)	0 violations on 20 pages, mobile and desktop
Browsers and devices (Playwright)	Chromium on Pixel 5 and desktop, Firefox desktop, WebKit on iPhone 13: all journeys pass
Bundle	route-level code splitting; core about 110 KB gzipped; charts (103 KB) load only for admins

Analysis: inference is a small share of the round trip, and limiting it to one thread costs about 5 ms, so a modest server CPU is sufficient. On a slow phone and network the wait rises to about 3 s, dominated by the upload; the offline queue means the

worker can continue with the next patient meanwhile. Low-end figures are emulated (Chrome DevTools); physical low-end devices and real iOS Safari have not been tested.

6. Model evaluation, limitations and next steps

Model (test split)	Threshold	Sensitivity	Specificity	Precision	AUC	Targets 85% / 80%
DR v2.0 (35,426 images)	0.418	0.851 (0.845-0.857)	0.748 (0.743-0.754)	0.706	0.900	sensitivity met, specificity not
Glaucoma (1,040 eyes, 42 positive)	0.320	0.762	0.721	0.103	0.818	not met
Cataract (1,044 eyes, 46 positive)	0.541	0.978	0.962	0.542	0.991	met (limited evidence)

DR v2 is MobileNetV2 fine-tuned from `block_13_expand` on the Kaggle EyePACS + APTOS + Messidor redistribution, split 70/15/15 by image group; the threshold was chosen on validation (highest with sensitivity ≥ 85%) and the test split scored once. Its figures are a **held-out test split by image (no patient IDs available; near-duplicates could not be excluded); not external validation; not clinically validated**: the notebook found as many groups as images, so augmented copies may sit in both training and test. Every figure was recomputed from the notebook's predictions with the app's own threshold and bootstrap code and matched; the served file has the notebook's checksum, and on 24 of its test photos the app reproduces the notebook's scores within 0.021 (tolerance 0.03, my choice) with no decision changed. Glaucoma and cataract (ODIR-5K) were split by patient; ODIR labels come from free-text keywords and its positive classes are small. For reference only, Gulshan et al. (2016) reported 90.3% / 98.1% on different data and a high-specificity operating point; our referable-DR (grade 2+) sensitivity is 0.869. The earlier DR model (v0.3) reached only 0.542 sensitivity on its validation fold.

- Specificity below target means about 25% of eyes without DR are referred, and about 9 in 10 glaucoma flags are false alarms; the result screens say so next to each result, and a specialist reviews every referral.
- Not done: on-device inference in the browser, a load balancer, a usability study with health workers (SUS kit ready), tests on a physical phone (planned on an iPhone 13 Pro after the HTTPS deploy), external validation and smartphone-lens images.

7. Use of AI assistance

I used Claude to help me build this project. This project was designed and directed by Michael Maina Kimani. I wrote the proposal, chose the scope and the honesty rules (real trained models only, no invented results), trained and evaluated the diabetic retinopathy model on Kaggle, and made the decisions on what to build and what to cut.

I used Claude (Anthropic), through Claude Code, to help write, refactor and test the backend and frontend code, the test suites and the documentation, working under my direction. I reviewed the results and I am responsible for the submitted work.